Department : Computer Science and Engineering

Course Title : Compiler Design Laboratory

Course Code : CSE 3212

Title : **Compiler Project Proposal**

Submission Date : 19 January, 2026

Submitted To,	**Submitted By,**
Md.Badiuzzaman Shuvo **Lecturer,** Department of Computer Science and Engineering Khulna University of Engineering and Technology **Subah Nawar** **Lecturer,** Department of Computer Science and Engineering Khulna University of Engineering and Technology	**Name: MD. Jahid Hasan Jim** **Roll** : 2107054 **Year** : 3rd **Term :** 2nd **Group**: **A2**

COMPILER PROJECT PROPOSAL
SPACE MISSION PROGRAMMING LANGUAGE (SMPL)

1. INTRODUCTION

The Space Mission Programming Language (SMPL) is a domain-specific, human-readable programming language that represents programming constructs using space exploration terminology. In SMPL, a program resembles a space mission where variables act as cargo items, operations act as spacecraft maneuvers, and control structures resemble mission protocols.

This project aims to design and implement a mini compiler for SMPL using Flex for lexical analysis and Bison for syntax analysis. The compiler translates SMPL source programs into equivalent C code, demonstrating the complete compilation process.

2. OBJECTIVES

- Design a creative and structured domain-specific language
- Implement lexical analysis using Flex
- Implement syntax analysis using Bison
- Define grammar rules for expressions and control structures
- Translate SMPL programs into equivalent C code
- Demonstrate core compiler design principles

3. LANGUAGE FEATURES

- Multiple data type support (integers, floats, characters, strings)
- Cargo-based variable declaration and initialization
- Assignment using keyword-based syntax
- Arithmetic and logical operations using spacecraft-action keywords
- Relational comparisons using mission-status keywords
- Conditional execution using mission-protocol conditions
- Iterative constructs including WHILE, FOR, and DO-WHILE loops
- Switch-case statements for multi-way branching
- Function declarations and calls
- Array support
- Input and output using communication metaphors
- Break and continue statements
- Explicit program start and termination markers

4. COMPLETE KEYWORDS AND SYMBOLS MAPPING

4.1 Program Structure

SMPL Keyword	C Equivalent	Description	Example
MISSION	main()	Program start	MISSION
LANDING	return 0;	Program end	LANDING;
LAUNCH	{	Block start	LAUNCH
ABORT	}	Block end	ABORT

4.2 Data Types

SMPL Keyword	C Equivalent	Description	Example
CARGO_INT	int	Integer type	CARGO_INT fuel;
CARGO_FLOAT	float	Float type	CARGO_FLOAT weight;
CARGO_CHAR	char	Character type	CARGO_CHAR code;
CARGO_DOUBLE	double	Double type	CARGO_DOUBLE dist;
VOID_SPACE	void	Void type	VOID_SPACE

4.3 Variable Declaration and Assignment

SMPL Keyword	C Equivalent	Description	Example
LOAD	Declaration	Variable declaration	LOAD CARGO_INT fuel;
STORE	=	Assignment operator	fuel STORE 100;

4.4 Arithmetic Operators

SMPL Keyword	C Equivalent	Description	Example
COMBINE	+	Addition	a STORE COMBINE a 5;
REDUCE	-	Subtraction	a STORE REDUCE a 3;
AMPLIFY	*	Multiplication	a STORE AMPLIFY a 2;
SPLIT	/	Division	a STORE SPLIT a 2;
REMAINDER	%	Modulus	a STORE REMAINDER a 3;
BOOST	++	Increment	BOOST a;
DEGRADE	--	Decrement	DEGRADE a;

4.5 Relational Operators

SMPL Keyword	C Equivalent	Description	Example
EXCEEDS	>	Greater than	fuel EXCEEDS 50
BELOW	<	Less than	oxygen BELOW 100
EXCEEDS_OR_EQUAL	>=	Greater or equal	fuel EXCEEDS_OR_EQUAL 50
BELOW_OR_EQUAL	<=	Less than or equal	oxygen BELOW_OR_EQUAL 100
MATCHES	==	Equal to	status MATCHES 1
DIFFERS	!=	Not equal	alert DIFFERS 0

4.6 Logical Operators

SMPL Keyword	C Equivalent	Description	Example
BOTH	&&	Logical AND	a EXCEEDS 5 BOTH b BELOW 10
EITHER	||	Logical OR	a MATCHES 0 EITHER b MATCHES 0
NEGATE	!	Logical NOT	NEGATE (a MATCHES 0)

4.7 Conditional Statements

SMPL Keyword	C Equivalent	Description	Example
CHECK_IF	if	If condition	CHECK_IF (fuel EXCEEDS 50)
ELSE_CHECK	else if	Else if condition	ELSE_CHECK (fuel EXCEEDS 20)
OTHERWISE	else	Else condition	OTHERWISE

4.8 Switch-Case Statement

SMPL Keyword	C Equivalent	Description	Example
PROTOCOL	switch	Switch statement	PROTOCOL (status)
SCENARIO	case	Case label	SCENARIO 1:
DEFAULT_PROTOCOL	default	Default case	DEFAULT_PROTOCOL:
TERMINATE	break	Break statement	TERMINATE;

4.9 Loop Constructs

SMPL Keyword	C Equivalent	Description	Example
ORBIT_WHILE	while	While loop	ORBIT_WHILE (fuel EXCEEDS 0)
SEQUENCE	for	For loop	SEQUENCE (...)
REPEAT	do	Do-while loop	REPEAT
UNTIL	while	Until condition	UNTIL (condition);

4.10 Loop Control

SMPL Keyword	C Equivalent	Description	Example
TERMINATE	break	Break loop	TERMINATE;
SKIP	continue	Continue loop	SKIP;

4.11 Functions

SMPL Keyword	C Equivalent	Description	Example
FUNCTION	function def	Function declaration	FUNCTION calculate
RETURN_CARGO	return	Return statement	RETURN_CARGO result;

4.12 Arrays

SMPL Keyword	C Equivalent	Description	Example
CARGO_ARRAY	array[]	Array declaration	CARGO_INT CARGO_ARRAY data[10];

4.13 Input/Output

SMPL Keyword	C Equivalent	Description	Example
RECEIVE	scanf	Input	RECEIVE fuel;
TRANSMIT	printf	Output	TRANSMIT fuel;

5. VARIABLE NAMING CONVENTIONS

Variables in SMPL represent cargo items or mission parameters.

Rules:

- Must begin with a letter or underscore
- Can contain letters, digits, and underscores
- Case-sensitive
- Reserved keywords cannot be used as identifiers

Examples:

```
LOAD CARGO_INT fuel STORE 100;
LOAD CARGO_FLOAT oxygen2 STORE 50.5;
LOAD CARGO_CHAR status STORE 'A';
```

6. OPERATORS IN DETAIL

6.1 Arithmetic Operations

```
LOAD CARGO_INT a STORE 10;
LOAD CARGO_INT b STORE 3;

a STORE COMBINE a b;        // a = a + b
a STORE REDUCE a b;          // a = a - b
a STORE AMPLIFY a b;         // a = a * b
a STORE SPLIT a b;           // a = a / b
a STORE REMAINDER a b;       // a = a % b
BOOST a;                     // a++
DEGRADE b;                   // b--
```

6.2 Relational and Logical Operations

```
CHECK_IF (a EXCEEDS b BOTH a BELOW 100) LAUNCH
    TRANSMIT a;
ABORT

CHECK_IF (status MATCHES 1 EITHER status MATCHES 2) LAUNCH
    TRANSMIT status;
ABORT
```

7. CONTROL STRUCTURES WITH EXAMPLES

7.1 If-Else Statement

```
CHECK_IF (fuel EXCEEDS 75) LAUNCH
    TRANSMIT "Fuel level high";
ABORT
ELSE_CHECK (fuel EXCEEDS 50) LAUNCH
    TRANSMIT "Fuel level medium";
ABORT
OTHERWISE LAUNCH
    TRANSMIT "Fuel level low";
ABORT
```

7.2 Switch-Case Statement

```
PROTOCOL (status) LAUNCH
    SCENARIO 1:
        TRANSMIT "Mission Active";
        TERMINATE;

    SCENARIO 2:
        TRANSMIT "Mission On Hold";
        TERMINATE;

    SCENARIO 3:
        TRANSMIT "Mission Completed";
        TERMINATE;

    DEFAULT_PROTOCOL:
        TRANSMIT "Unknown Status";
        TERMINATE;
ABORT
```

7.3 While Loop

```
ORBIT_WHILE (fuel EXCEEDS 0) LAUNCH
    TRANSMIT fuel;
    fuel STORE REDUCE fuel 1;
ABORT
```

7.4 For Loop

```
SEQUENCE (LOAD CARGO_INT i STORE 0; i BELOW 10; BOOST i) LAUNCH
    TRANSMIT i;
ABORT
```

7.5 Do-While Loop

```
REPEAT LAUNCH
    TRANSMIT counter;
    BOOST counter;
ABORT UNTIL (counter BELOW 5);
```

7.6 Nested Loops with Break and Continue

```
SEQUENCE (LOAD CARGO_INT i STORE 0; i BELOW 5; BOOST i) LAUNCH
    CHECK_IF (i MATCHES 2) LAUNCH
        SKIP;
    ABORT

    CHECK_IF (i MATCHES 4) LAUNCH
        TERMINATE;
    ABORT

    TRANSMIT i;
ABORT
```

8. FUNCTIONS

8.1 Function Declaration and Call

```
FUNCTION CARGO_INT calculate(CARGO_INT x, CARGO_INT y) LAUNCH
    LOAD CARGO_INT result STORE COMBINE x y;
    RETURN_CARGO result;
ABORT

MISSION
    LOAD CARGO_INT sum STORE calculate(5, 10);
    TRANSMIT sum;
LANDING;
```

8.2 Void Function

```
FUNCTION VOID_SPACE display() LAUNCH
    TRANSMIT "Hello from Space Mission";
ABORT
```

9. ARRAYS

9.1 Array Declaration and Usage

```
LOAD CARGO_INT CARGO_ARRAY numbers[5] STORE {1, 2, 3, 4, 5};

SEQUENCE (LOAD CARGO_INT i STORE 0; i BELOW 5; BOOST i) LAUNCH
    TRANSMIT numbers[i];
ABORT
```

10. COMPLETE SAMPLE SMPL PROGRAM

```
MISSION
    // Variable declarations
    LOAD CARGO_INT fuel STORE 100;
    LOAD CARGO_INT counter STORE 0;
    LOAD CARGO_FLOAT temperature STORE 25.5;
    LOAD CARGO_INT CARGO_ARRAY scores[3] STORE {85, 90, 78};

    // For loop
    TRANSMIT "Countdown:";
    SEQUENCE (LOAD CARGO_INT i STORE 10; i EXCEEDS 0; DEGRADE i) LAUNCH
        TRANSMIT i;
    ABORT

    // While loop with condition
    ORBIT_WHILE (fuel EXCEEDS 0 BOTH fuel BELOW_OR_EQUAL 100) LAUNCH
        CHECK_IF (fuel MATCHES 50) LAUNCH
            TRANSMIT "Half fuel remaining";
        ABORT

        fuel STORE REDUCE fuel 10;

        CHECK_IF (fuel BELOW 20) LAUNCH
            TERMINATE;
        ABORT
    ABORT

    // Do-while loop
    REPEAT LAUNCH
        TRANSMIT counter;
        BOOST counter;
    ABORT UNTIL (counter BELOW 5);

    // Switch statement
```

```
    LOAD CARGO_INT status STORE 2;
    PROTOCOL (status) LAUNCH
        SCENARIO 1:
            TRANSMIT "Status: Active";
            TERMINATE;

        SCENARIO 2:
            TRANSMIT "Status: Standby";
            TERMINATE;

        DEFAULT_PROTOCOL:
            TRANSMIT "Status: Unknown";
            TERMINATE;
    ABORT

    // Array processing
    LOAD CARGO_INT total STORE 0;
    SEQUENCE (LOAD CARGO_INT j STORE 0; j BELOW 3; BOOST j) LAUNCH
        total STORE COMBINE total scores[j];
    ABORT
    TRANSMIT total;

    // Conditional with logical operators
    CHECK_IF (temperature EXCEEDS 20.0 BOTH temperature BELOW 30.0) LAUNCH
        TRANSMIT "Temperature is comfortable";
    ABORT
    OTHERWISE LAUNCH
        TRANSMIT "Temperature is extreme";
    ABORT

LANDING;
```

11. EQUIVALENT C CODE

```
#include <stdio.h>

int main() {
    // Variable declarations
    int fuel = 100;
    int counter = 0;
    float temperature = 25.5;
    int scores[3] = {85, 90, 78};

    // For loop
    printf("Countdown:\n");
    for (int i = 10; i > 0; i--) {
        printf("%d\n", i);
```

```
}

// While loop with condition
while (fuel > 0 && fuel <= 100) {
    if (fuel == 50) {
        printf("Half fuel remaining\n");
    }

    fuel = fuel - 10;

    if (fuel < 20) {
        break;
    }
}

// Do-while loop
do {
    printf("%d\n", counter);
    counter++;
} while (counter < 5);

// Switch statement
int status = 2;
switch (status) {
    case 1:
        printf("Status: Active\n");
        break;

    case 2:
        printf("Status: Standby\n");
        break;

    default:
        printf("Status: Unknown\n");
        break;
}

// Array processing
int total = 0;
for (int j = 0; j < 3; j++) {
    total = total + scores[j];
}
printf("%d\n", total);

// Conditional with logical operators
if (temperature > 20.0 && temperature < 30.0) {
    printf("Temperature is comfortable\n");
}
else {
```

```
            printf("Temperature is extreme\n");
        }

        return 0;
    }
```

12. IMPLEMENTATION PLAN

Phase 1: Lexical Analysis (Flex)

- Tokenization of all keywords (50+ keywords)
- Recognition of identifiers, constants, and operators
- Handling of data types (int, float, char, double)
- String and character literal recognition
- Comment handling (single-line and multi-line)

Phase 2: Syntax Analysis (Bison)

- Grammar rules for all data type declarations
- Grammar for keyword-based assignments and operators
- Grammar for arithmetic, relational, and logical expressions
- Grammar for if-else, switch-case structures
- Grammar for all loop types (while, for, do-while)
- Grammar for function declarations and calls
- Grammar for array declarations and access
- Grammar for break, continue, and return statements

Phase 3: Semantic Analysis

- Type checking for variables
- Scope management for variables and functions
- Array bounds checking
- Function signature matching

Phase 4: Code Generation

- Direct translation from SMPL to C code
- Proper mapping of all keywords to C equivalents
- Maintaining correct syntax and structure

13. GRAMMAR SUMMARY

Basic Program Structure

```
    program → MISSION statement_list LANDING
    statement_list → statement | statement statement_list
    statement → declaration | assignment | control_structure | io_statement
```

Declarations

```
declaration → LOAD data_type identifier STORE value
data_type → CARGO_INT | CARGO_FLOAT | CARGO_CHAR | CARGO_DOUBLE
```

Expressions

```
expression → term | expression operator term
operator → COMBINE | REDUCE | AMPLIFY | SPLIT | REMAINDER
relational_expr → expression rel_op expression
rel_op → EXCEEDS | BELOW | MATCHES | DIFFERS |
         EXCEEDS_OR_EQUAL | BELOW_OR_EQUAL
```

14. EXPECTED OUTCOME

- A fully working mini compiler for SMPL
- Support for all major C programming constructs
- Correct parsing and translation into valid C code
- Error detection and reporting
- Demonstration of all compiler phases
- Test cases covering all language features

15. CONCLUSION

The Space Mission Programming Language (SMPL) provides a comprehensive yet creative approach to compiler design by replacing all traditional C programming constructs with meaningful, domain-specific keywords inspired by space exploration. This enhanced version includes all major C language features including multiple data types, all loop types, switch-case statements, functions, arrays, and logical operators.

Through the use of Flex and Bison, this project demonstrates complete lexical analysis, syntax analysis, semantic analysis, and code generation in a clear and structured manner. The language's extensive feature set and well-defined grammar make it an excellent comprehensive compiler project for academic evaluation.